

ANALISI TECNICA DEL PROGETTO

Piattaforma di Visite Nutrizionistiche

Questa sezione descrive quali tecnologie vengono usate, come vengono usate e come sarà organizzata la struttura del progetto, mantenendo Spring Boot come backend principale e Flask come backend secondario di supporto.

1. Obiettivo dell'Analisi Tecnica

L'analisi tecnica descrive come verrà realizzato il progetto dal punto di vista architetturale e implementativo. In questa parte si definiscono le tecnologie utilizzate, il ruolo di ciascun componente, la struttura del progetto e il modo in cui i moduli collaborano tra loro.

2. Stack Tecnologico

Il progetto si avvale delle seguenti tecnologie:

- **Frontend:** React con TypeScript.
- **Backend Principale:** Spring Boot.
- **Backend Secondario:** Flask.
- **Database Relazionale:** PostgreSQL (dati applicativi).
- **Database NoSQL:** MongoDB (log di sistema).
- **Infrastruttura:** Docker e Docker Compose.

3. Struttura Architetturale Generale

L'architettura è di tipo distribuito ma controllato. Il frontend comunica esclusivamente con Spring Boot, che gestisce sicurezza, validazioni e logica di business. Flask viene richiamato solo da Spring per funzioni specializzate. PostgreSQL salva i dati funzionali e MongoDB salva i log del sistema.

4. Ruolo dei Componenti

Frontend (React & TypeScript)

Gestisce l'interfaccia utente, le dashboard e le prenotazioni. React organizza la UI in componenti riutilizzabili, mentre TypeScript garantisce un codice tipizzato e chiaro.

Backend Principale (Spring Boot)

È il cuore del sistema. Gestisce autenticazione, ruoli, flussi di prenotazione e la ricerca geolocalizzata. Coordina le chiamate verso Flask e salva i dati finali.

Backend Secondario (Flask)

Usato per elaborazioni specializzate (es. ranking o suggerimenti slot) trasmesse da Spring. Non espone funzionalità dirette al frontend.

5. Gestione Dati e Persistenza

Tecnologia	Ruolo nel Progetto
PostgreSQL	Dati strutturati: utenti, nutrizionisti, disponibilità e prenotazioni.
MongoDB	Log di autenticazione, errori di sistema e audit delle operazioni sensibili.

6. Autenticazione e Sicurezza

L'autenticazione è realizzata in Spring Boot con **JWT** e password cifrate. I ruoli (ADMIN, CLIENT, NUTRITIONIST) determinano l'accesso agli endpoint e alle pagine del frontend tramite Spring Security.

7. Logica delle Prenotazioni e Posizione

Il sistema raccoglie dati geografici per ordinare i professionisti per distanza. La prenotazione viene validata da Spring (verificando disponibilità e conflitti) prima del salvataggio finale su database.

8. Struttura del Codice

Organizzazione Modulare:

- **Frontend:** Diviso in components, pages, services, hooks e types.
- **Spring:** Organizzato in controller, service, repository, entity e dto.
- **Flask:** Moduli semplici per routes e servizi di calcolo.

9. Flusso Tecnico tra i Componenti

Il frontend invia una richiesta a Spring Boot. Spring verifica permessi e dati, eventualmente interroga Flask per calcoli specifici, salva i risultati su PostgreSQL, registra l'operazione su MongoDB e restituisce la risposta alla UI.

10. DevOps e Deployment

L'intero progetto è containerizzato con **Docker Compose**. Sono previsti container separati per ogni servizio, configurati tramite variabili d'ambiente, con documentazione API e build automatica inclusa.